

沈阳航空航天大学

《模式识别》实验报告

题目：K-均值聚类算法

院（系）：人工智能学院

专 业：物联网工程

班 级：物联网2202

学 号：223428010210

姓 名：陈梓欣

指导教师：刘翠微

1 任务描述

A. 实验目的：通过编程和上机实验加深对无监督学习的理解和认识，掌握基于模型的聚类算法 K 均值算法的设计与实现；了解软件绘图的基本方法，掌握如何调用相关函数绘制二维平面图。

B. 实验环境：PC 机，window 操作系统，python（包括 numpy、matplotlib、os 库）。

C. 实验要求：

- (1) 掌握 K 均值聚类算法的原理；
- (2) 编程实现 K 均值聚类算法，并将实验结果显示出来。

2 实验内容

(1) 数据集的生成与保存

使用 `numpy.random.multivariate_normal` 函数，分别生成三组二维高斯正态分布数据：

- 第一组参数：均值 (0,0)，协方差矩阵 $\begin{bmatrix} 0.9 & 0.4 \\ 0.4 & 0.3 \end{bmatrix}$ ，样本数 50。
- 第二组参数：均值 (5,0)，协方差矩阵相同，样本数 50。
- 第三组参数：均值 (3,3)，协方差矩阵相同，样本数 50。

将三组数据合并为一个 150×2 的数据集。

调用 `numpy.savetxt` 函数将数据保存为 `data.txt` 文件，每行存储一个二维坐标。

使用 `matplotlib.pyplot.scatter` 绘制数据点分布，直观展示数据集的整体情况。

(2) K-means 聚类算法实现

随机初始化聚类中心：

从数据集中随机选择 $K=3$ 个点作为初始聚类中心。

迭代过程：

- 计算每个样本点到各个聚类中心的欧氏距离；
- 将样本点分配到距离最近的簇；
- 更新每个簇的中心为该簇所有点的均值；
- 重复迭代，直到聚类中心不再明显变化或达到最大迭代次数。

可视化结果：

- 使用不同颜色绘制不同类别的样本点；
- 聚类中心用特殊符号标注（如五角星、十字或圆圈）。

(3) 固定 $K=3$ 多次运行程序

由于 K-means 聚类的初始中心是随机选择的，因此每次运行可能得到不同的聚类结果。

在 $K=3$ 的情况下，多次运行程序，分别绘制结果，以比较不同随机初始化下聚类结果的差异。

(4) 改变聚类个数 K

在相同数据集上，分别设置 $K=2$ 和 $K=4$ ，运行聚类程序，并绘制结果。

对比分析不同 K 值下的聚类效果与结果差异：

- $K=2$ ：可能会将三类数据压缩为两类，导致部分簇合并。
- $K=3$ ：基本能与真实分布对应，效果较好。
- $K=4$ ：可能会将某一类数据分裂为两个小簇，出现“过分聚类”现象。

3 实验结果和分析

A. 实验结果

(1) 原始数据分布

预先设置随机种子为 23，生成的 150 个二维数据点大致分布在三个区域，分别以 $(0, 0)$ 、 $(5, 0)$ 、 $(3, 3)$ 附近为中心，符合设定的高斯分布特性，绘制原始数据分布散点图如图 1 所示，数据文件保存结果如图 2 所示。

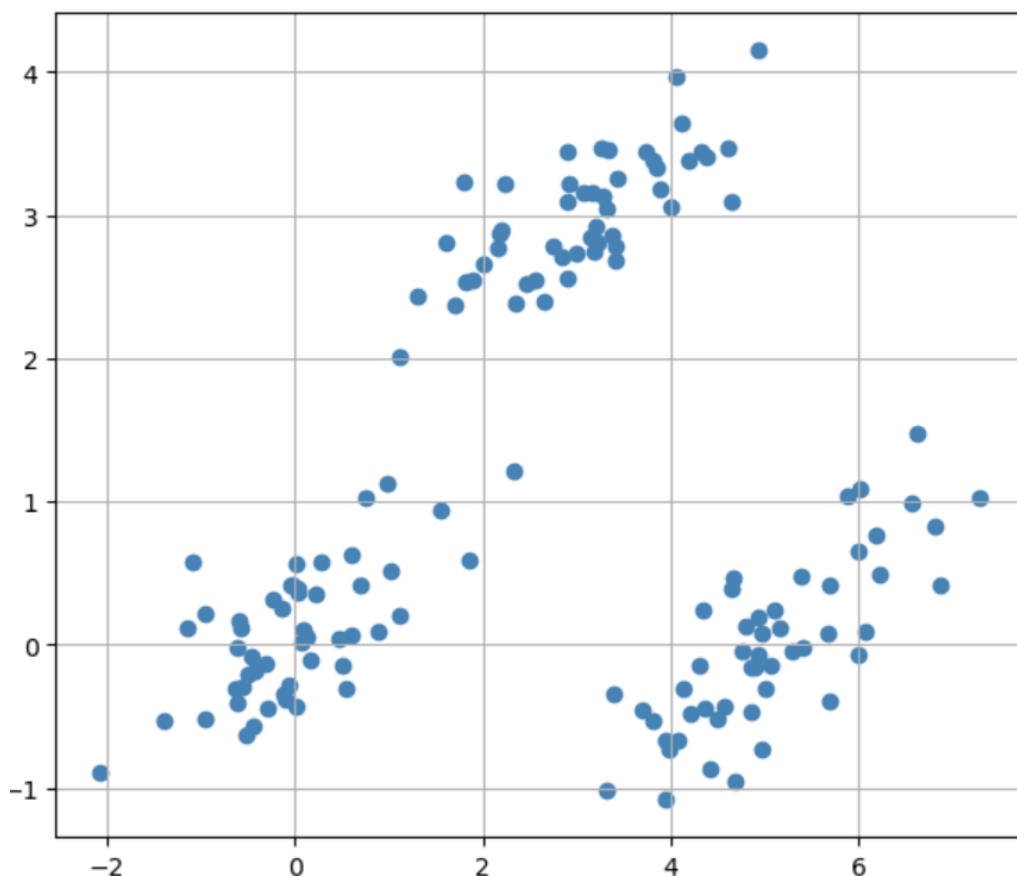


图 1 原始数据分布散点图

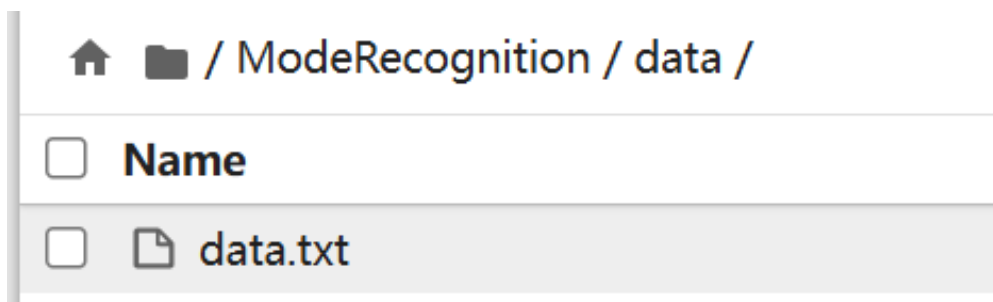


图 2 数据文件保存结果

(2) K=3 多次运行结果

多次运行结果表明，虽然 K-means 聚类的初始中心是随机选取的，但最终结果都能较好地将数据划分为三类。

聚类中心（红色星形点）收敛到数据簇的几何中心，说明 K=3 能够较好地反映数据的真实结构，聚类结果如图 3 所示。

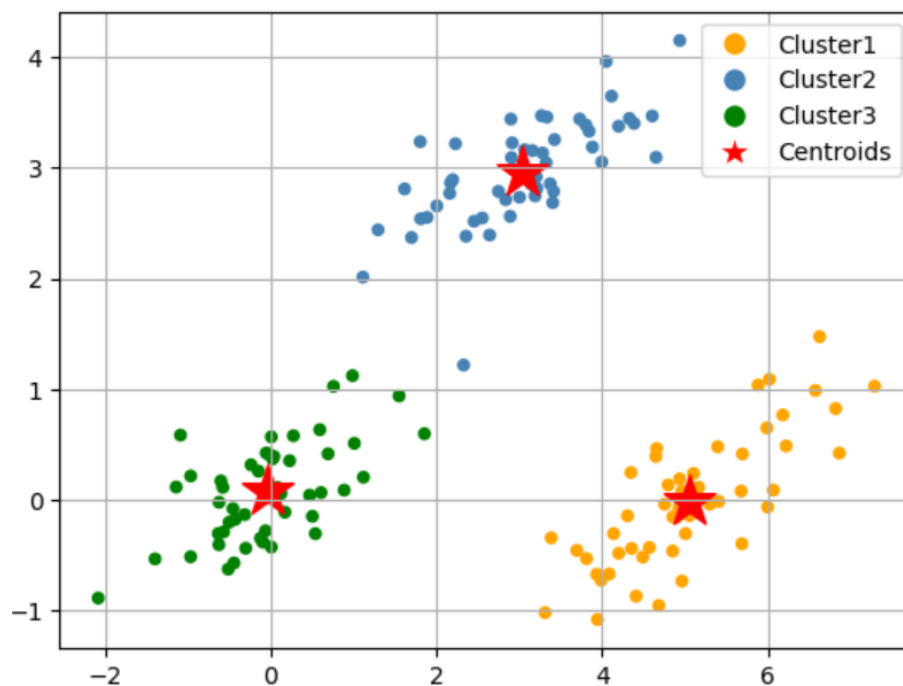


图3 K=3 多次运行聚类结果

不同运行结果在边界点分配上可能略有差异，但整体聚类效果一致。

(3) K=2 聚类结果

聚类算法将三个簇压缩为两个类别，其中相距较近的两个簇被错误合并，聚类结果如图4所示。

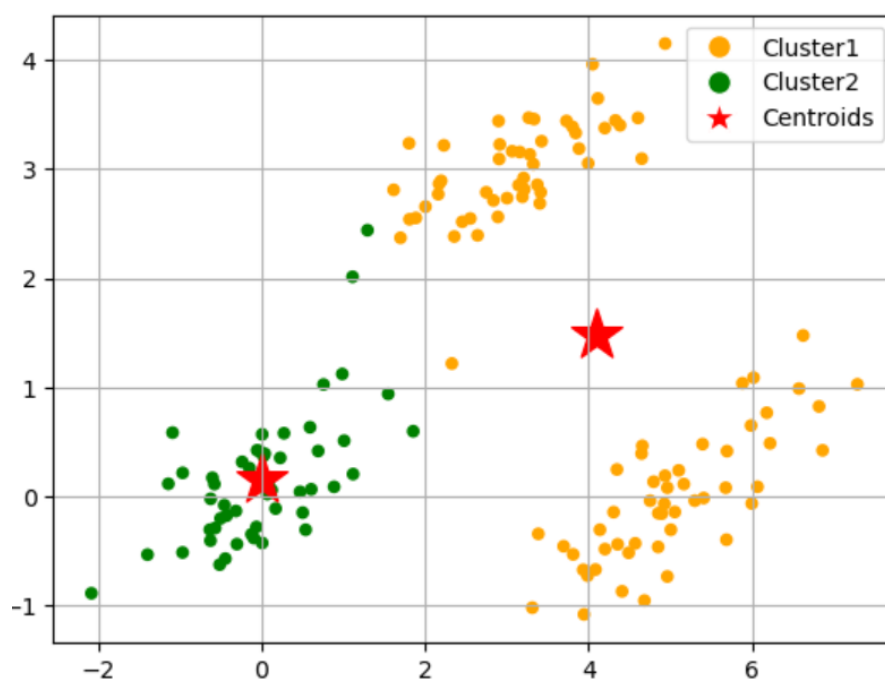


图4 K=2 多次运行聚类结果

这说明 K 值过小会导致“欠聚类”，不能正确反映数据的真实结构。

(4) $K=4$ 聚类结果

聚类结果显示原本三个簇被分成四类，其中某个簇被过分细分为两个小簇，聚类结果如图 5 所示。

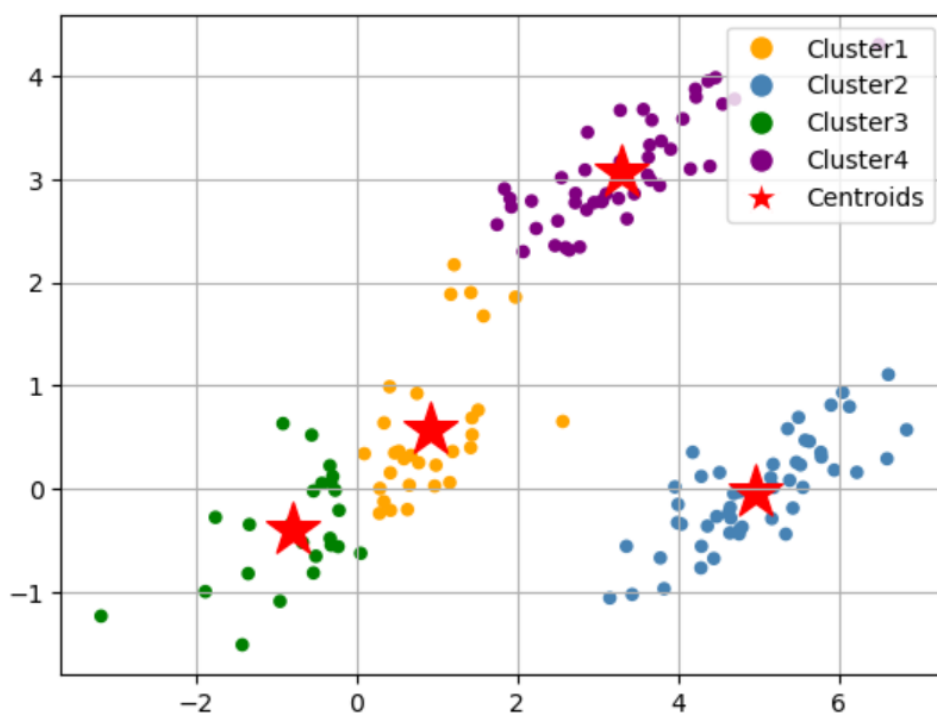


图 5 $K=4$ 多次运行聚类结果

这说明 K 值过大时容易产生“过分聚类”，使得同一类数据被错误划分。

B. 实验分析

(1) 聚类效果与 K 值的关系

当 K 与数据的真实簇数一致时（本实验中 $K=3$ ），聚类效果最好。

当 K 值偏小，聚类结果会将多个真实簇合并，导致分类不准确；

当 K 值偏大，聚类结果会将单个簇分裂，出现不必要的小簇。

(2) 聚类中心初始化的影响

K -means 聚类对初始聚类中心敏感，不同随机初始化可能导致不同的局部最优解。

(3) 实验结论

K-means 聚类在样本数适中、簇形态接近球状时效果良好；

实验中 K=3 与真实数据分布一致，聚类效果最优；

通过对比 K=2 与 K=4 的结果，验证了选择合适 K 值对聚类效果的重要性。

4 附录

导入库

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib as mpl
```

```
import matplotlib.patches as mpatches
```

```
import scipy.io
```

设置随机种子

```
np.random.seed(23)
```

定义高斯分布参

```
mean1 = [0, 0]
```

```
mean2 = [5, 0]
```

```
mean3 = [3, 3]
```

```
cov = [[0.9, 0.4],  
       [0.4, 0.3]]
```

每组 50point

```
data1 = np.random.multivariate_normal(mean1, cov, 50)
```

```
data2 = np.random.multivariate_normal(mean2, cov, 50)
```

```
data3 = np.random.multivariate_normal(mean3, cov, 50)
```

合并数据，150 个点

```
data = np.vstack((data1, data2, data3))
```

```
X = data
```

```
# 保存到 data.txt 文件 每行一个二维点
```

```
np.savetxt("data/data.txt", X, fmt="%.6f")
```

```
# 读取文件 绘制散点图
```

```
plt.figure(figsize=(7, 6))
```

```
plt.scatter(X[:, 0], X[:, 1], c="steelblue")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# 随机初始化质心
```

```
def kMeansInitCentroids(X, k):
```

```
    #从 X 的数据中随机取 k 个作为质心
```

```
    index = np.random.randint(0, len(X)-1, k)
```

```
    return X[index]
```

```
# 计算数据点到质心的距离，并判断该数据点属于哪个质心
```

```
def findClosestCentroids(X, centroids):
```

```
    #idx 中数据表明对应 X 的数据是属于哪一个质心的
```

```
    idx = np.zeros(len(X)).reshape( X.shape[0], -1)
```

```
    for i in range(len(X)):
```

```
        minDistance = float('inf'); #初始无限大
```

```
        index = 0
```

```
        for k in range(len(centroids)):
```

```
            #距离计算，使用欧几里得距离
```

```
            distance = np.sum(np.power(X[i]-centroids[k], 2))
```

```
            if(distance < minDistance):
```

```
                minDistance = distance
```

```
                index = k
```

```
        idx[i] = index
```

```
    return idx
```

```
# 重新计算质心
```

```
def computeCentroids(X, idx):
```

```
    k = set(np.ravel(idx).tolist()) #找到所有聚类中心索引
```



```

k = list(k)
centroids = np.ndarray((len(k),X.shape[1]))
for i in range(len(k)):
    #选择数据 X 中类别为 k[i]的数据

    data = X[np.where(idx==k[i])[0]]

    #重新计算聚类中心

    centroids[i] = np.sum(data,axis=0)/len(data)
return centroids

## 合成 k-means 算法（限制迭代次数版）

# def k_means(X, k, max_iters):
#     initial_centroids = kMeansInitCentroids(X,k)
#     #迭代

#     for i in range(max_iters):
#         if i==0:
#             centroids = initial_centroids

#         #计算样本到质心的距离，并返回每个样本所属的质心

#         idx = findClosestCentroids(X, centroids)

#         #重新计算聚类中心

#         centroids = computeCentroids(X, idx)
#     return idx,centroids

# idx,centroids = k_means(X, 3, 8)
## print(idx)
# print(centroids)

# 合成 k-means 算法（不限制迭代次数版，人工迭代）

def k_means(X, k):
    initial_centroids = kMeansInitCentroids(X,k)
    centroids = initial_centroids

    #计算样本到质心的距离，并返回每个样本所属的质心

    idx = findClosestCentroids(X, centroids)

    #重新计算聚类中心

    centroids = computeCentroids(X, idx)
    return idx,centroids

idx,centroids = k_means(X, 3)

```

```

# print(idx)
print(centroids)

# 绘制聚类后散点图

from matplotlib.lines import Line2D
cm_dark = mpl.colors.ListedColormap(['orange', 'steelblue', 'green'])

plt.scatter(X[:, 0], X[:, 1], c=np.ravel(idx), cmap=cm_dark, s=20)
plt.scatter(centroids[:, 0], centroids[:, 1], c="red", marker='*', s=500)

# 构造图例： 前三个 Cluster 用圆点， Centroids 用五角星

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label='Cluster1',
           markerfacecolor='orange', markersize=10),
    Line2D([0], [0], marker='o', color='w', label='Cluster2',
           markerfacecolor='steelblue', markersize=10),
    Line2D([0], [0], marker='o', color='w', label='Cluster3',
           markerfacecolor='green', markersize=10),
    Line2D([0], [0], marker='*', color='w', label='Centroids',
           markerfacecolor='red', markersize=15)
]

plt.legend(handles=legend_elements, loc='upper right')
plt.grid(True)
plt.show()

```